



SpecTokenizer: A Lightweight Streaming Codec in the Compressed Spectrum Domain

Zixiang Wan^{1,2}, Guochang Zhang¹, Yifeng He¹, Jianqiang Wei^{1*}

¹Audio Innovation Technology Department, Anker Inc, China

²School of Computer Science, Beijing University of Posts and Telecommunications, China

wzx802@bupt.edu.cn, {baron.zhang, nolan.he, alex.wei}@anker-in.com

Abstract

Neural Audio Codecs (NACs) have gained growing attention in recent years as technologies for audio compression and audio representation in speech language models. While mainstream NACs typically require G-level computation and M-level parameters, the performance of lightweight and streaming NACs remains underexplored. This paper proposes SpecTokenizer, a lightweight streaming codec that operates in the compressed spectral domain. Composed solely of alternating CNN and RNN layers, SpecTokenizer achieves greater efficiency and better representational capability through multi-scale modeling in the compressed spectrum domain. At 4 kbps, the proposed SpecTokenizer achieves comparable or superior performance compared to the codec with state-of-the-art lightweight architecture while requiring only 20% of the computation and 10% of the parameters. Furthermore, it significantly outperforms the codec when using similar computational and storage resources.

Index Terms: neural audio codec, lightweight, streaming, single codebook, compressed spectrum domain

1. Introduction

Neural Audio Codec (NAC) compresses audio signals into sequences of discrete codes for efficient storage and transmission [1–5]. Recently, NAC has gained growing attention as a key technology in Speech Language Modeling (SLM) [6–8]. By converting continuous audio signals into discrete codes, Large Language Models (LLMs)—which have achieved great success in text processing—can now be applied to various speech processing tasks [9–11].

Many high-performance NAC methods have been proposed, optimized for higher audio quality, bitrate efficiency, and lower computational cost [12–15]. However, most of these models have G-level computational complexity and M-level parameter counts, making them unsuitable for deployment on edge hardware. The performance of models with lower computational and parameter scales has not been thoroughly explored. On the other hand, in signal-level tasks like speech enhancement [16], speech signal improvement [17] and music source separation [18], spectrogram-based models often perform better. Yet, in audio codecs, spectrogram-based codecs often underperform compared to waveform-based codecs. To bridge these gaps, we propose SpecTokenizer.

When NACs are used for audio reconstruction and as tokenizers in SLMs, the following features are particularly important:

- **Audio Quality:** Critical for NAC performance, impacting users and tasks. NACs should faithfully reconstruct audio details without distortion, vital for applications like speech communication and modeling.

- **Bitrate Efficiency:** Refers to transmitting information efficiently per unit of bitrate. A NAC with high bitrate efficiency can achieve comparable or superior audio quality at a lower bitrate.
- **Streaming:** To achieve real-time interaction with simultaneous speaking and responding, codecs need streaming support for low-latency speech encoding and response generation.
- **Single Codebook:** Single-codebook models are favored over multi-codebook models since the latter introduce additional design complexity to the SLM architecture [19–21].
- **Low Computational Complexity:** NACs with low computational complexity enable faster encoding and decoding, reducing computational demands and allocating more resources to SLMs.
- **Low Parameter Count:** NACs with fewer parameters require less storage, reducing memory demands and freeing up resources for edge hardware.
- **Low Token Rate:** Longer sequences can slow down and destabilize LLM training. Therefore, it's preferable to use NAC models with a low token rate to support SLMs.

This paper introduces SpecTokenizer, a lightweight streaming codec in the compressed spectral domain—the first neural audio codec with M-level computation and K-level parameters. Composed solely of alternating CNN and RNN layers, SpecTokenizer achieves efficiency and enhanced representational capability through multi-scale modeling in the compressed spectrum domain. SpecTokenizer offers streaming capability, low computational and memory demands, a wide bitrate range, and a single codebook design, while maintaining high audio quality. Under similar computational and storage resources, SpecTokenizer significantly improves audio quality compared to state-of-the-art codecs. Furthermore, after model miniaturization, SpecTokenizer achieves comparable or superior performance using only 20% of the computation and 10% of the parameters.

2. Related work

In recent years, most well-known neural audio codecs have been based on vector quantization (VQ) networks [22], such as SoundStream [12], EnCodec [13], and DAC [14]. Among them, SoundStream is the first universal codec handling multiple types of audio. EnCodec improves compression rates by integrating a lightweight Transformer language model in the discrete latent space. Building on this, researchers [14] have explored quantization dropout, a bitrate scalable technique. They have also shown that periodic inductive bias functions outperform common activation functions in audio signal modeling. These models process audio waveforms directly and are therefore categorized as waveform-based codecs.

In contrast, spectrogram-based codecs focus on processing more intuitive audio spectral features. For example, Lyra [23] converts audio waveforms into log mel-spectrograms and directly quantizes them into tokens. Due to the irreversibility of mel-spectrograms, Lyra relies on a vocoder to synthesize waveforms. To avoid the inefficiency of heavy vocoders, codecs like TFNet [24] and ESC [25] use the invertible Short-Time Fourier Transform (STFT) to convert waveforms into complex spectra. This design allows the reconstructed STFT spectra to be losslessly converted back into waveforms via the inverse STFT. FreqCodec [15] introduces depthwise and grouped convolutions into the encoder and decoder. This significantly reduces computational and parameter complexity while maintaining speech quality comparable to state-of-the-art codecs at the time.

3. Method

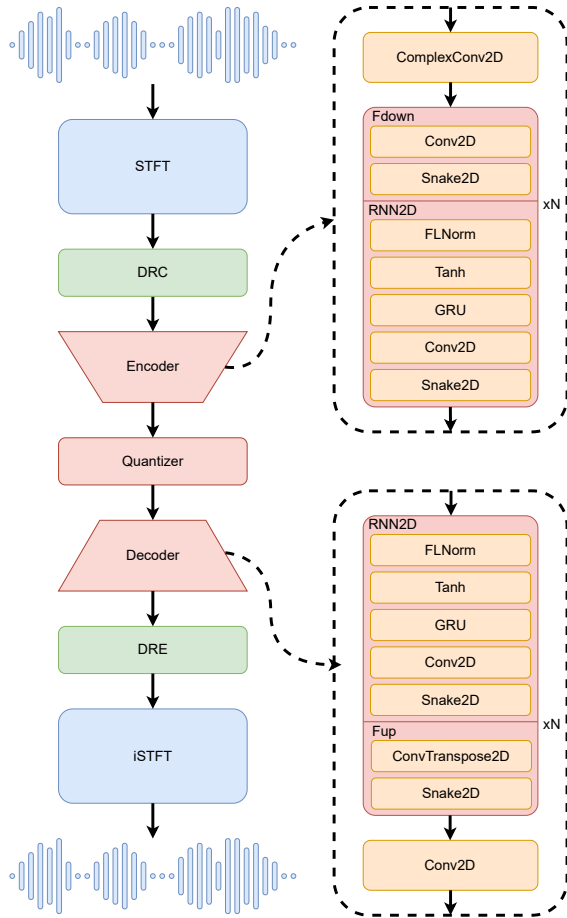


Figure 1: The architecture of the proposed SpecTokenizer. **DRC** denotes dynamic range compression, **DRE** denotes dynamic range expansion, and **FLNorm** denotes frame-wise layer normalization.

3.1. Overview

Figure 1 shows the architecture of SpecTokenizer. The speech signal x is transformed into a complex spectrogram s via STFT,

followed by **Dynamic Range Compression (DRC)** to obtain s_c . The encoder E extracts latent features z from s_c , and the quantization layer Q generates the compressed representation z_q . The decoder G reconstructs the compressed spectrum $\hat{s}_c$. After **Dynamic Range Expansion (DRE)** to get $\hat{s}$, the inverse STFT (iSTFT) is applied to obtain the reconstructed audio signal $\hat{x}$.

3.2. Spectral Compression

Experience shows that audio signals typically have extremely high dynamic ranges in amplitude, which poses challenges for directly processing raw spectral data. To address this, we perform **spectral compression** on the spectrum. The core idea is to apply a compression function to the amplitude while preserving the phase.

In the dynamic range compression stage, given a complex spectrum s , we compress it as:

$$s_c = f(|s|) \cdot \text{sign}(s), \quad \text{where } f(|s|) = |s|^{1/p} \quad (1)$$

Here, $|s|$ denotes the magnitude of s , $\text{sign}(s) = s/|s|$ represents its phase, and $p > 0$ is the compression coefficient.

In the dynamic range expansion stage, after obtaining the reconstructed compressed spectrum $\hat{s}_c$ from the model, we apply the inverse function f^{-1} to convert the compressed-scale spectrum back into the linear-scale spectrum:

$$\hat{s} = f^{-1}(|\hat{s}_c|) \cdot \text{sign}(\hat{s}_c), \quad \text{where } f^{-1}(|s_c|) = |s_c|^p \quad (2)$$

3.3. Encoder and Decoder

Our model’s encoder and decoder architecture is inspired by MTFAA-Net [16]. The encoder E starts with a 2D complex convolution, followed by N alternating frequency down-sampling blocks (*Fdown*) and RNN2D blocks. Each *Fdown* block contains a 2D strided convolution with channel number C , kernel size K , stride S , and the Snake2D activation function. Each RNN2D block includes a **Frame-wise Layer Normalization (FLNorm)**, a Tanh activation function, a GRU layer, a 2D convolution with kernel size and stride set to 1, and the Snake2D activation function [26].

RNNs are suitable for tasks with short historical dependencies like audio encoding and decoding, and they have low memory consumption during inference, making them ideal for lightweight edge deployment. The frame-wise layer normalization is specifically designed for RNNs, featuring streaming processing, stable training, and accelerated convergence. It normalizes over the channel and frequency dimensions at each time step:

$$\mu_t = \frac{1}{C \times F} \sum_{c=1}^C \sum_{f=1}^F Y_{c,t,f} \quad (3)$$

$$\hat{Y}_{c,t,f} = \frac{Y_{c,t,f} - \mu_t}{\sqrt{\sigma_t^2 + \epsilon}} \quad (4)$$

where $Y_{c,t,f}$ is the input tensor; C and F are the channel and frequency dimensions; μ_t and σ_t^2 are the mean and variance over C and F at time t ; and ϵ is a small constant to prevent division by zero.

The Snake2D activation function captures periodic features in the data. The decoder mirrors the encoder’s structure but uses transposed convolutions instead of strided convolutions, with strides opposite to those of the encoder.

Table 1: Comparison of SpecTokenizer and baseline codecs.

Codec	Streaming	VQ Type	Domain
SpecTokenizer (Ours)	✓	RVQ / Single	Compressed Complex
EnCodec [13]	✓	RVQ	Waveform
HiFi-Codec [29]	✗	GRVQ	Waveform
DAC [14]	✗	RVQ	Waveform
FreqCodec [15]	✓	RVQ	Magnitude, Phase
SpeechTokenizer [30]	✗	RVQ	Waveform
WavTokenizer [31]	✗	Single	Waveform

3.4. Discriminator

We use two discriminators: the Multi-Period Discriminator (MPD) from HiFi-GAN [27], and the frequency-domain Multi-Band Multi-Scale STFT Discriminator (MBMS-STFT) from DAC [14], which improves phase modeling in practice.

3.5. Loss Function

The loss function of SpecTokenizer is written as follows:

$$\mathcal{L} = \lambda_{\text{rec}} \mathcal{L}_{\text{rec}} + \lambda_{\text{adv}} \mathcal{L}_{\text{adv}} + \lambda_{\text{feat}} \mathcal{L}_{\text{feat}} + \lambda_{\text{cmt}} \mathcal{L}_{\text{cmt}} \quad (5)$$

where λ_{rec} , λ_{adv} , λ_{feat} , and λ_{cmt} are weight coefficients for the reconstruction loss $\mathcal{L}_{\text{rec}}$, adversarial loss $\mathcal{L}_{\text{adv}}$, feature matching loss $\mathcal{L}_{\text{feat}}$, and commitment loss $\mathcal{L}_{\text{cmt}}$, respectively.

3.6. Single Large Codebook

In the single large codebook setting, we initialize the codebook using a clustering method and update it via exponential moving average (EMA) [22]. To improve codebook utilization, we introduce factorized codebook lookup [14] and expiration mechanism [28]. Since the codebook size is much larger than the batch data volume—leading to high repetition in batch-sampled data—we add a data pool module. This module accumulates historical data on a larger scale to better represent the data distribution.

4. Experiments

4.1. Dataset

We used LibriTTS [32] as the training dataset, containing approximately 585 hours of English speech. For evaluation, we used the datasets specified by the Codec-SUPERB challenge at SLT 2024 [5]. Codebook utilization, UTMOS [33] and XLSR-MOS [34] were evaluated using the LibriTTS test set.

4.2. Baselines

We selected state-of-the-art (SOTA) codec models as baselines for SpecTokenizer. To ensure fair comparisons, we employed the official weights provided by the EnCodec [13], HiFi-Codec [29], DAC [14], FreqCodec [15], SpeechTokenizer [30], and WavTokenizer [31]. The differences between SpecTokenizer and these models are shown in Table 1.

4.3. Evaluation Metrics

To comprehensively evaluate our codec, we followed the methods proposed by the Codec-SUPERB challenge at SLT 2024

Due to a bug in the widely used EnCodec code, the expiration mechanism has not been effectively used in many codecs. See <https://github.com/facebookresearch/encodec/issues/61>

[5], analyzing the quality of reconstructed audio from both application-level and signal-level perspectives. At the application level, we focused on Automatic Speech Recognition (ASR). At the signal level, we used objective metrics such as PESQ, STOI, SDR, and Mel spectrum loss to quantify audio quality, ensuring the model’s performance in preserving speech content and signal fidelity. Additionally, we included codebook utilization, UTMOS and XLSR-MOS.

4.4. Implementation Details

Our model accepts audio input at 16 kHz. For spectral compression, the coefficient p is set to 2. For the encoder and decoder, we set the number of blocks $N = 4$, the number of channels $C = [192, 256, 384, 800, 1280]$, convolution kernel sizes $K = [(3, 4), (3, 4), (3, 4), (3, 4)]$, and strides $S = [(1, 4), (1, 4), (1, 4), (2, 4)]$. Each element of K and S corresponds to the time and frequency dimensions, respectively. Downsampling results in a frame rate of 50 frames per second. For the quantizer, we adopt a Residual Vector Quantizer (RVQ) composed of 12 codebooks of size 1024, with dropout set to 0.5. For the MPD and MBMS-STFT discriminators, we maintain the same structure as in DAC. In the single large codebook setting, we set the number of clustering iterations to 50, the EMA decay to 0.99, the expiration threshold to 0.01, and the data pool size to $\max(32768, \text{codebook size})$. We train a separate SpecTokenizer model for each bitrate from scratch.

4.5. Main Results

4.5.1. Results at 6 kbps

Table 2 (a) shows the results at a bitrate of 6 kbps. SpecTokenizer achieves the best performance across WER, PESQ, STOI, SDR, MelLoss, UTMOS and XLSR-MOS metrics, with SDR significantly higher than other codecs. This indicates that at 6 kbps, the speech reconstructed by SpecTokenizer has excellent intelligibility, fidelity, and naturalness, closely resembling the original speech.

4.5.2. Results at 4 kbps

Table 2 (b) shows the results at 4 kbps. We observe the following: (1) SpecTokenizer still performs best on all metrics, with SDR significantly higher than other codecs. Compared to 6 kbps, the advantages of SpecTokenizer over other codecs at 4 kbps are more pronounced. (2) When the bitrate decreases from 6 kbps to 4 kbps, DAC’s performance drops significantly, while FreqCodec and SpecTokenizer maintain relatively good performance.

4.5.3. Results at 0.5 kbps

Table 2 (c) shows the results at 0.5 kbps. We observe the following: (1) Among codecs using RVQ, when the bitrate decreases to 0.5 kbps, DAC and SpeechTokenizer’s performance drops significantly, while FreqCodec and SpecTokenizer still maintain relatively good performance. (2) Under the single large codebook setting, SpecTokenizer achieves the best PESQ, STOI, and MelLoss metrics, and ranks second on WER, SDR, UTMOS and XLSR-MOS. (3) SpeechTokenizer performs poorly on PESQ, STOI, SDR, UTMOS and XLSR-MOS, but relatively well on WER, possibly because it uses semantic distillation during training, allowing the first codebook’s tokens to encode sufficient content information. (4) WavTokenizer performs well on UTMOS and XLSR-MOS but poorly on PESQ,

Table 2: Comparison of different codec models at various bitrates. (a) Rows 1–6; (b) Rows 7–11; (c) Rows 12–17. The SpecTokenizer* indicates SpecTokenizer with a single large codebook of size 32k.

Codec	Bitrate (kbps)	WER(%)↓	PESQ↑	STOI↑	SDR↑	MelLoss↓	UTMOS↑	XLSR-MOS↑
Ground Truth	-	2.87	-	-	-	-	4.04	4.08
EnCodec	12	3.28	3.07	0.943	10.14	0.929	3.51	3.97
EnCodec	6	3.63	2.47	0.909	7.48	1.033	3.04	3.60
DAC	6	3.13	3.45	0.948	1.03	0.566	3.97	4.05
FreqCodec	6	3.39	2.63	0.821	3.98	0.885	3.84	3.97
SpecTokenizer	6	2.74	3.47	0.957	10.47	0.531	4.01	4.06
HiFi-Codec	4	5.62	2.05	0.866	3.50	1.652	3.57	3.70
SpeechTokenizer	4	4.34	1.98	0.843	1.52	0.842	3.87	3.94
DAC	4	3.55	2.47	0.902	0.11	0.719	3.43	3.77
FreqCodec	4	3.44	2.46	0.814	3.14	0.917	3.79	3.92
SpecTokenizer	4	2.89	3.04	0.935	8.36	0.601	3.98	4.04
WavTokenizer	0.5	19.97	1.39	0.740	-9.98	1.112	3.57	3.87
SpeechTokenizer	0.5	10.63	1.16	0.572	-14.58	1.813	1.26	2.83
DAC	0.5	78.82	1.08	0.583	-7.93	1.946	1.25	1.35
FreqCodec	0.5	22.57	1.48	0.718	-2.02	1.246	2.97	3.14
SpecTokenizer	0.5	20.32	1.44	0.755	-3.06	1.009	2.90	3.19
SpecTokenizer*	0.5	14.82	1.51	0.767	-2.42	0.957	3.10	3.47

Table 3: Comparison of cost and performance of different codec models at 4 kbps. For the mini version of SpecTokenizer, we set $C = [16, 24, 32, 64, 96]$.

Codec	FLOPs (G)	Params (M)	PESQ	SDR
DAC	55.66	74.06	2.47	0.11
FreqCodec	2.18	4.50	2.46	3.14
HiFi-Codec	53.62	62.56	2.05	3.50
SpeechTokenizer	17.09	103.68	1.98	1.52
SpecTokenizer (mini)	0.43	0.45	2.51	6.02
SpecTokenizer (base)	56.40	70.61	3.04	8.36

Table 4: Results of the ablation study on our proposed codec.

Codec	WER(%)	PESQ	SDR	MelLoss	UTMOS
SpecTokenizer	2.74	3.47	10.47	0.531	4.01
- DRC&DRE	3.11	3.39	10.26	0.775	3.95
- FLNorm	2.87	3.39	10.24	0.550	3.96
- Snake2D	2.81	3.15	8.88	0.577	3.95
- Multi-Scale	3.19	3.31	9.69	0.594	3.84

SDR, and MelLoss, indicating that while the reconstructed speech is natural, its fidelity is poor.

4.5.4. Results of Model Miniaturization

We conducted model miniaturization experiments at 4 kbps. As shown in Table 3, the miniaturized SpecTokenizer performs comparably or better than FreqCodec while using only 20% of the computational complexity and 10% of the parameters. Compared to DAC, SpecTokenizer achieves similar or superior performance using only 0.8% of DAC’s computational complexity and 0.6% of its parameters. This demonstrates the advantages of spectrogram-based models: they require significantly less computational complexity and fewer parameters than waveform-based models because modeling in the frequency domain is easier, as confirmed by other studies [15, 35].

Table 5: Results of the ablation study on the improvements to the single large codebook setup of SpecTokenizer.

Codec	WER(%)	PESQ	UTMOS	Utilization Rate(%)
SpecTokenizer	10.94	1.63	3.13	62.0
+ expirecode	9.76	1.65	3.25	93.4
+ datapool	8.81	1.72	3.48	94.8

4.6. Ablation Studies

To further investigate the role of each component in our model, we conducted ablation studies on the architecture by examining the effects of: not performing spectral compression; replacing FLNorm with BatchNorm; replacing Snake2D with ReLU; and eliminating multi-scale modeling by keeping RNN2D only at the largest scale. The experimental results are shown in Table 4. We found that removing any component reduced model performance to varying degrees. Specifically, replacing Snake2D caused the most severe drop in PESQ, indicating that Snake2D significantly contributes to audio reconstruction quality.

To investigate the effects of each improvement in the single codebook setup, we conducted experiments with a codebook size of 32k, frame rate of 50 fps, and bitrate of 750 bps. Table 5 shows the effects of each improvement in the single codebook setup of SpeechTokenizer. These improvements are also applicable in other settings.

5. Conclusion

This paper introduces SpecTokenizer, which is the first attempt to develop a neural audio codec (NAC) model with M-level computation and K-level parameters. It features streaming capability, low computational demand, low memory usage, wide bitrate range, and a single codebook design. Compared to the state-of-the-art lightweight codec FreqCodec, SpecTokenizer achieves comparable or better performance using only 20% of the computation and 10% of the parameters, and performs significantly better under similar computational and storage conditions. SpecTokenizer opens up a new direction for lightweight and efficient streaming NAC.

6. References

- [1] Y. Guo, Z. Li, H. Wang, B. Li, C. Shao, H. Zhang, C. Du, X. Chen, S. Liu, and K. Yu, "Recent Advances in Discrete Speech Tokens: A Review," *arXiv preprint arXiv:2502.06490*, 2025.
- [2] H. Wu, H.-L. Chung *et al.*, "Codec-SUPERB: An in-depth analysis of sound codec models," in *Findings of ACL 2024*. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 10 330–10 348. [Online]. Available: <https://aclanthology.org/2024.findings-acl.616>
- [3] J. Shi, J. Tian, Y. Wu, J.-w. Jung, J. Q. Yip, Y. Masuyama, W. Chen, Y. Wu, Y. Tang, M. Baali *et al.*, "ESPnet-Codec: Comprehensive training and evaluation of neural codecs for audio, music, and speech," *arXiv preprint arXiv:2409.15897*, 2024.
- [4] H. Wu, N. Kanda, S. E. Eskimez, and J. Li, "TS3-Codec: Transformer-Based simple streaming single codec," *arXiv preprint arXiv:2411.18803*, 2024.
- [5] H. Wu, X. Chen, Y.-C. Lin, K. Chang, J. Du, K.-H. Lu, A. H. Liu, H.-L. Chung, Y.-K. Wu, D. Yang *et al.*, "Codec-SUPERB@ SLT 2024: A lightweight benchmark for neural audio codec models," *arXiv preprint arXiv:2409.14085*, 2024.
- [6] J. Yao, H. Liu, C. Chen, Y. Hu, E. Chng, and L. Xie, "GenSE: Generative speech enhancement via language models using hierarchical modeling," *arXiv preprint arXiv:2502.02942*, 2025.
- [7] H. Guo, F. Xie, K. Xie, D. Yang, D. Guo, X. Wu, and H. Meng, "SoCodec: A semantic-ordered multi-stream speech codec for efficient language model based text-to-speech synthesis," in *2024 IEEE Spoken Language Technology Workshop (SLT)*. IEEE, 2024, pp. 645–651.
- [8] S. Ji, M. Fang, Z. Jiang, S. Zheng, Q. Chen, R. Huang, J. Zuo, S. Wang, and Z. Zhao, "Language-Codec: Reducing the gaps between discrete codec representation and speech language models," *arXiv preprint arXiv:2402.12208*, 2024.
- [9] D. Wang, M. Cui, D. Yang, X. Chen, and H. Meng, "A comparative study of discrete speech tokens for Semantic-Related tasks with large language models," *arXiv preprint arXiv:2411.08742*, 2024.
- [10] H. Wu, X. Chen, Y.-C. Lin, K.-w. Chang, H.-L. Chung, A. H. Liu, and H.-y. Lee, "Towards Audio Language Modeling-An Overview," *arXiv preprint arXiv:2402.13236*, 2024.
- [11] R. Huang, M. Li, D. Yang, J. Shi, X. Chang, Z. Ye, Y. Wu, Z. Hong, J. Huang, J. Liu *et al.*, "AudioGPT: Understanding and generating speech, music, sound, and talking head," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 21, 2024, pp. 23 802–23 804.
- [12] N. Zeghidour, A. Luebs, A. Omran, J. Skoglund, and M. Tagliasacchi, "SoundStream: An end-to-end neural audio codec," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 30, pp. 495–507, 2021.
- [13] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi, "High fidelity neural audio compression," *arXiv preprint arXiv:2210.13438*, 2022.
- [14] R. Kumar, P. Seetharaman, A. Luebs, I. Kumar, and K. Kumar, "High-fidelity audio compression with improved RVQGAN," *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [15] Z. Du, S. Zhang, K. Hu, and S. Zheng, "FunCodec: A fundamental, reproducible and integrable open-source toolkit for neural speech codec," in *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2024, pp. 591–595.
- [16] G. Zhang, L. Yu, C. Wang, and J. Wei, "Multi-scale temporal frequency convolutional network with axial attention for speech enhancement," in *ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2022, pp. 9122–9126.
- [17] N.-C. Ristea, B. Naderi, A. Saabas, R. Cutler, S. Braun, and S. Branets, "ICASSP 2024 speech signal improvement challenge," *IEEE Open Journal of Signal Processing*, 2025.
- [18] W.-T. Lu, J.-C. Wang, Q. Kong, and Y.-N. Hung, "Music source separation with band-split rope Transformer," in *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2024, pp. 481–485.
- [19] D. Yang, H. Guo, Y. Wang, R. Huang, X. Li, X. Tan, X. Wu, and H. Meng, "UniAudio 1.5: Large language model-driven audio codec is a few-shot audio task learner," *arXiv preprint arXiv:2406.10056*, 2024.
- [20] Z. Du, Y. Wang, Q. Chen, X. Shi, X. Lv, T. Zhao, Z. Gao, Y. Yang, C. Gao, H. Wang *et al.*, "CosyVoice 2: Scalable streaming speech synthesis with large language models," *arXiv preprint arXiv:2412.10117*, 2024.
- [21] D. Yang, J. Tian, X. Tan, R. Huang, S. Liu, X. Chang, J. Shi, S. Zhao, J. Bian, X. Wu *et al.*, "UniAudio: An audio foundation model toward universal audio generation," *arXiv preprint arXiv:2310.00704*, 2023.
- [22] A. Van Den Oord, O. Vinyals *et al.*, "Neural discrete representation learning," *Advances in neural information processing systems*, vol. 30, 2017.
- [23] W. B. Kleijn, A. Storus, M. Chinen, T. Denton, F. S. Lim, A. Luebs, J. Skoglund, and H. Yeh, "Generative speech coding with predictive variance regularization," in *ICASSP 2021-2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2021, pp. 6478–6482.
- [24] X. Jiang, X. Peng, C. Zheng, H. Xue, Y. Zhang, and Y. Lu, "End-to-end neural speech coding for real-time communications," in *ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2022, pp. 866–870.
- [25] Y. Gu and E. Diao, "ESC: Efficient speech coding with cross-scale residual vector quantized transformers," *arXiv preprint arXiv:2404.19441*, 2024.
- [26] L. Ziyin, T. Hartwig, and M. Ueda, "Neural networks fail to learn periodic functions and how to fix it," *Advances in Neural Information Processing Systems*, vol. 33, pp. 1583–1594, 2020.
- [27] J. Kong, J. Kim, and J. Bae, "HiFi-GAN: Generative adversarial networks for efficient and high fidelity speech synthesis," *Advances in neural information processing systems*, vol. 33, pp. 17 022–17 033, 2020.
- [28] P. Dhariwal, H. Jun, C. Payne, J. W. Kim, A. Radford, and I. Sutskever, "Jukebox: A generative model for music," *arXiv preprint arXiv:2005.00341*, 2020.
- [29] D. Yang, S. Liu, R. Huang, J. Tian, C. Weng, and Y. Zou, "HiFi-Codec: Group-residual vector quantization for high fidelity audio codec," *arXiv preprint arXiv:2305.02765*, 2023.
- [30] X. Zhang, D. Zhang, S. Li, Y. Zhou, and X. Qiu, "SpeechTokenizer: Unified speech tokenizer for speech large language models," *arXiv preprint arXiv:2308.16692*, 2023.
- [31] S. Ji, Z. Jiang, W. Wang, Y. Chen, M. Fang, J. Zuo, Q. Yang, X. Cheng, Z. Wang, R. Li *et al.*, "WavTokenizer: an efficient acoustic discrete codec tokenizer for audio language modeling," *arXiv preprint arXiv:2408.16532*, 2024.
- [32] H. Zen, V. Dang, R. Clark, Y. Zhang, R. J. Weiss, Y. Jia, Z. Chen, and Y. Wu, "LibriTTS: A corpus derived from librispeech for text-to-speech," *arXiv preprint arXiv:1904.02882*, 2019.
- [33] T. Saeki, D. Xin, W. Nakata, T. Koriyama, S. Takamichi, and H. Saruwatari, "UTMOS: Utokyo-sarulab system for voicemos challenge 2022," *arXiv preprint arXiv:2204.02152*, 2022.
- [34] B. Tamm, R. Vandenbergh, and H. Van Hamme, "Analysis of xls-r for speech quality assessment," in *2023 IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*. IEEE, 2023, pp. 1–5.
- [35] X.-H. Jiang, Y. Ai, R.-C. Zheng, H.-P. Du, Y.-X. Lu, and Z.-H. Ling, "MDCTCodec: A lightweight mdct-based neural audio codec towards high sampling rate and low bitrate scenarios," in *2024 IEEE Spoken Language Technology Workshop (SLT)*. IEEE, 2024, pp. 540–547.